Lista Computacional II

Impa Tech

Otimização Linear - Semestre Letivo 2026/2

Integrantes: Kenzo Nagamine, Samuel Leite, Lucas Caracas.

1 Introdução

Um labirinto pode ser visto como um problema de caminho mínimo. Neste trabalho partimos da *imagem* de um labirinto, construímos automaticamente um grafo que representa seus movimentos possíveis e encontramos o menor caminho entre a entrada e a saída com o algoritmo de Dijkstra. Toda a implementação foi feita no GNU Octave (versão 11.3).

O fluxo do programa é:

1. ler a imagem e transformá-la em uma matriz binária (Seção 2);
2. transformar a matriz em um grafo de $20 \times 20 = 400$ vértices (Seção 3);
3. verificar e desenhar o grafo sobre o labirinto (Seção 4);
4. formular o problema de caminho mínimo (Seção 5);
5. executar Dijkstra e reconstruir o caminho (Seções 6 e 7);
6. desenhar o caminho sobre a imagem (Seção 8).

O código está organizado em cinco arquivos: `main.m` (script principal, que executa todas as partes), `id_no.m`, `coordenada_no.m`, `dijkstra.m` e `reconstruir_caminho.m`.

2 Da imagem à matriz binária (Parte 1)

2.1 Leitura e binarização

A imagem é lida com `imread`; se vier em RGB, é convertida para escala de cinza com `rgb2gray`. Em seguida aplicamos um valor de corte entre preto e branco (0,5 na escala $[0, 1]$, equivalente a 128 em 8 bits):

$$B(i,j) = \begin{cases} 1, & \text{se o pixel pertence a uma região livre (corredor, branco),} \\ 0, & \text{se o pixel pertence a uma parede (preto).} \end{cases}$$

Ou seja, o valor 1 marca onde se pode andar e o valor 0 marca onde há parede.

2.2 Recorte e redução para 41×41

A imagem original tem uma margem branca ao redor do labirinto. Essa margem foi removida recortando a imagem na caixa que contém todos os pixels pretos. Sem esse recorte, a margem branca era confundida com abertura na borda, e a busca por entrada e saída devolvia uma faixa muito grande de pixels.

Depois do recorte, a imagem ainda tem centenas de pixels por lado, e cada corredor ou parede ocupa vários pixels. Como o labirinto é uma grade de 20×20 células, ele pode ser descrito

por $2 \cdot 20 + 1 = 41$ blocos por lado: os blocos de índice *ímpar* representam as paredes (e as aberturas entre células) e os de índice *par* representam as células. Reduzimos a matriz para 41×41 amostrando o pixel central de cada bloco.

Para escolher a resolução, testamos reduzir a imagem para 20, 30, 40, 50, 60 e 80 pixels por lado e plotamos os resultados (Figura 1). Com 20×20 quase nada do labirinto sobrevive à redução, e com 30×30 e 40×40 paredes e corredores se misturam. A partir de 50 pixels por lado o labirinto fica reconhecível. Adotamos 41×41, que é exatamente o tamanho dado pela conta acima ($2N + 1$) e representa cada parede e cada célula por um bloco.

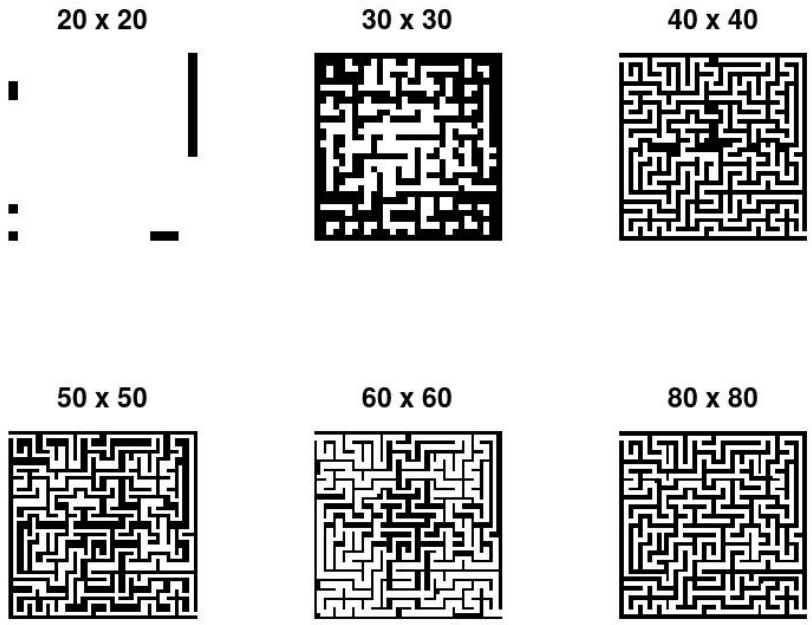


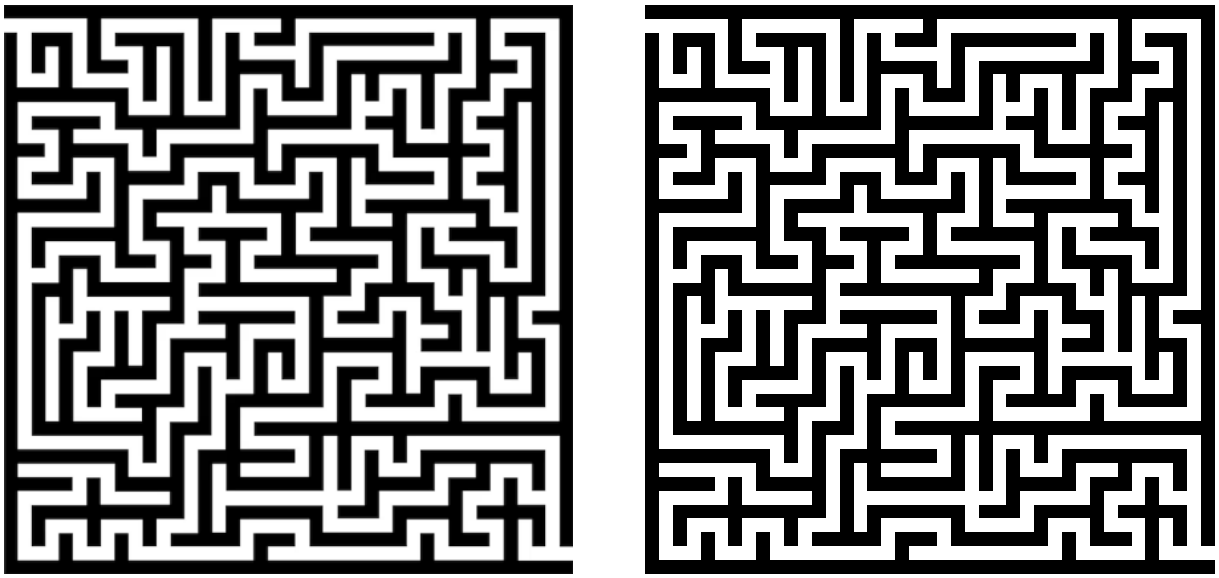
Figura 1: Teste com diferentes resoluções: o labirinto reduzido para 20×20, 30×30, 40×40, 50×50, 60×60 e 80×80 pixels.

2.3 Entrada e saída

A entrada e a saída são as aberturas na borda externa do labirinto, isto é, os pixels livres ($B = 1$) nas bordas da matriz, sem contar os cantos. O programa procura esses pixels nas quatro bordas e exige encontrar exatamente dois. Por convenção, a abertura mais à esquerda é a entrada e a outra é a saída. Para o labirinto fornecido:

- entrada: pixel $(2, 1)$, na borda esquerda, que corresponde à célula $(1, 1)$;
- saída: pixel $(40, 41)$, na borda direita, que corresponde à célula $(20, 20)$.

Como as células ficam nos índices pares, o pixel $(2i, \cdot)$ pertence à linha de células i, e daí vem a correspondência acima.



(a) Imagem original (recortada).

(b) Imagem binária 41×41 (branco = 1, preto = 0).

Figura 2: Imagem original e matriz binária obtida.

3 Da imagem ao grafo (Parte 2)

Vértices. Cada célula é um vértice, o que dá $20 \cdot 20 = 400$ vértices. O identificador da célula na linha i e coluna j é

$$\mathrm{id}(i, j) = (i - 1)\, n + j, \qquad n = 20,$$

implementado em `id_no.m`.

Coordenadas dos centros. A célula (i, j) corresponde ao bloco $(2i, 2j)$ da matriz 41×41. Para uma imagem de a linhas e l colunas, o centro da célula na imagem é

$$x = \tfrac{1}{2} + \left(2j - \tfrac{1}{2}\right)\frac{l}{41}, \qquad y = \tfrac{1}{2} + \left(2i - \tfrac{1}{2}\right)\frac{a}{41},$$

calculado por `coordenada_no.m` (que também devolve i e j a partir do identificador). Aqui x é a coluna e y é a linha, como nos eixos do `imshow`.

Arestas. Duas células vizinhas são ligadas se *não há parede entre elas*. Como as paredes e passagens ficam nos blocos de índice ímpar, basta olhar o pixel que separa as duas células:

- horizontal: $(i, j) \leftrightarrow (i, j + 1)$ existe se $B(2i,\, 2j + 1) = 1$;
- vertical: $(i, j) \leftrightarrow (i + 1, j)$ existe se $B(2i + 1,\, 2j) = 1$.

Só consideramos movimentos nas quatro direções, sem diagonais, e cada movimento tem custo $c_{uv} = 1$. Percorrendo todas as células uma vez (testando apenas a direita e abaixo), obtemos cada aresta uma única vez, e o conjunto E é guardado como lista $[u\ v]$. Para o Dijkstra, montamos a matriz de adjacência esparsa A *simétrica*: $A(u, v) = A(v, u) = 1$ para toda aresta $\{u, v\}$, já que o grafo é não direcionado.

Resultado. O grafo obtido tem

$$|V| = 400 \quad \text{vértices} \qquad \text{e} \qquad |E| = 399 \quad \text{arestas.}$$

4 Visualização do grafo (Parte 3)

O grafo foi desenhado sobre a própria imagem: cada vértice no centro de sua célula e cada aresta ligando duas células com passagem livre. A entrada aparece em verde e a saída em azul. Para conferir que nenhuma aresta atravessa parede, além da inspeção visual, o programa testa automaticamente cada aresta: todos os pixels de B entre as duas células devem ser livres. O resultado foi **0 arestas atravessando parede**.

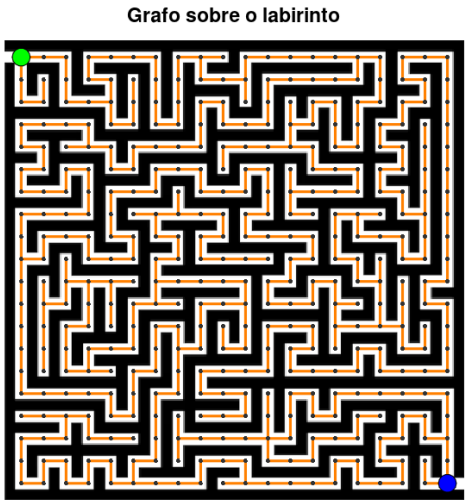


Figura 3: Grafo sobre o labirinto: vértices (pontos), arestas (laranja), entrada (verde) e saída (azul).

5 Formulação como problema de caminho mínimo (Parte 4)

Seja $G = (V, E)$ o grafo construído. O vértice de entrada é $s = \mathrm{id}(1, 1) = 1$ e o de saída é $t = \mathrm{id}(20, 20) = 400$.

Por que é um problema de caminho mínimo. Cada célula é um vértice e cada passagem livre é uma aresta. Ir da entrada à saída andando pelo labirinto é o mesmo que percorrer um caminho $P = (s, v_1, \ldots, v_k, t)$ em G. Entre todos os caminhos possíveis, queremos o que minimiza

$$C(P) = \sum_{e \in P} c_e,$$

que é exatamente um problema de caminho mínimo de s a t.

Significado do custo. Como $c_e = 1$ para toda aresta, o custo do caminho é $C(P) = |P| - 1$, onde $|P|$ é o número de vértices do caminho. Assim, $C(P)$ é o número de movimentos entre células vizinhas, e minimizar o custo é encontrar a rota com o menor número de movimentos.

6 Algoritmo de Dijkstra (Parte 5)

A função `[dist, anterior] = dijkstra(A, s)` recebe a matriz de adjacência A (com $A(u, v) > 0$ igual ao custo da aresta $u \to v$) e o vértice inicial s, e devolve:

- `dist(v)`: a distância mínima de s até v (∞ se v não é alcançável);
- `anterior(v)`: o predecessor de v em um caminho mínimo (0 se não há).

O algoritmo mantém um conjunto de vértices *fechados*, cuja distância já é definitiva:

1. inicialize $\text{dist}(s) = 0$, $\text{dist}(v) = \infty$ para $v \neq s$ e $\text{anterior}(v) = 0$; nenhum vértice está fechado;
2. escolha o vértice u *aberto* com menor $\text{dist}(u)$; se essa distância for ∞, pare (o restante é inalcançável);
3. feche u;
4. para cada vizinho aberto v de u, se $\text{dist}(u) + c_{uv} < \text{dist}(v)$, atualize $\text{dist}(v) \leftarrow \text{dist}(u) + c_{uv}$ e $\text{anterior}(v) \leftarrow u$ (*relaxamento*);
5. volte ao passo 2 enquanto houver vértices abertos.

O algoritmo é correto porque os custos são não negativos: quando u é fechado, nenhum caminho que passe por vértices ainda abertos pode chegar a u com custo menor. Nossa implementação escolhe o menor dist varrendo o vetor, o que custa $O(|V|^2)$; com $|V| = 400$ isso é mais que suficiente. Como todos os custos são iguais a 1, o resultado coincide com o de uma busca em largura, mas o código serve para custos quaisquer. Testamos a função também em um grafo pequeno com pesos diferentes, em que o caminho indireto de custo 2 vence a aresta direta de custo 5.

7 Reconstrução do caminho e resultados (Parte 6)

Depois do Dijkstra, a função `caminho = reconstruir_caminho(anterior, s, t)` parte de t e segue os predecessores,

$$t,\ p(t),\ p(p(t)),\ \ldots$$

até alcançar s, e depois inverte a sequência. Se t não fosse alcançável, a função devolveria vazio. Os resultados para o labirinto fornecido estão na Tabela 1.

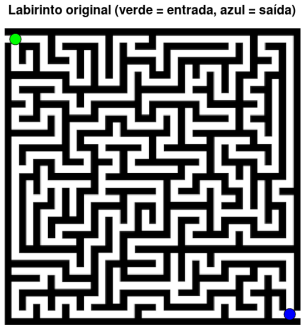
Grandeza	Valor
Custo do caminho mínimo $C(P)$	122
Número de movimentos	122
Número de vértices no caminho	123
Tempo do Dijkstra (uma execução)	0,0154 s
Tempo do Dijkstra (média de 50 execuções)	0,0132 s

Tabela 1: Resultados do caminho mínimo de $s = 1$ a $t = 400$.

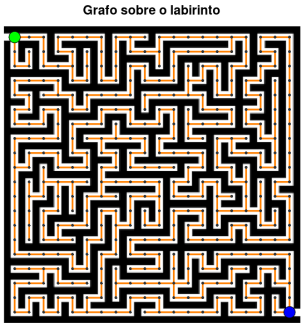
O custo e o número de movimentos coincidem, como previsto por $C(P) = |P| - 1$ ($123 - 1 = 122$). O tempo mede apenas a execução do Dijkstra, sem contar a leitura da imagem nem a construção do grafo.

8 Caminho mínimo sobre a imagem (Parte 7)

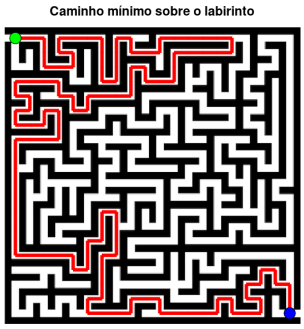
As figuras a seguir usam as coordenadas dos vértices: entrada em verde, saída em azul e caminho mínimo em vermelho.



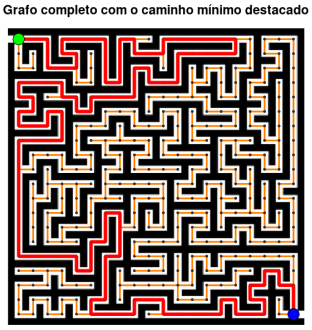
(a) Labirinto original, com entrada e saída.



(b) Grafo construído sobre o labirinto.



(c) Apenas o caminho mínimo.



(d) Grafo completo com o caminho mínimo destacado.

Figura 4: Visualizações finais: (a) labirinto, (b) grafo, (c) caminho mínimo e (d) grafo com o caminho destacado.

9 Análise dos resultados

- **O grafo é uma árvore.** O grafo tem 400 vértices e 399 arestas ($|E| = |V| - 1$), e o Dijkstra a partir de s alcança todos os 400 vértices, isto é, o grafo é conexo. Um grafo conexo com $|V| - 1$ arestas é uma árvore, portanto não há ciclos: entre quaisquer duas células existe um único caminho sem repetição. Isso é típico de um labirinto "perfeito" e explica por que o caminho mínimo de s a t é simplesmente a única rota entre eles.
- **O caminho é bem mais longo que a distância em linha reta.** Entre as células $(1, 1)$ e $(20, 20)$, a distância na grade seria $19+19 = 38$ movimentos, mas o caminho mínimo tem 122: o labirinto força voltas e desvios. O caminho passa por 123 das 400 células, cerca de 31% do total.
- **Consistência.** O custo (122) coincide com o número de movimentos, nenhuma aresta atravessa parede (0 arestas inválidas) e o caminho desenhado em vermelho segue os corredores brancos da entrada até a saída, o que confere visualmente com a imagem.
- **Tempo.** O Dijkstra levou cerca de 0,013 s a 0,015 s. O custo $O(|V|^2)$ com $|V| = 400$ é pequeno; para labirintos muito maiores seria melhor usar uma fila de prioridade (complexidade $O(|E| \log |V|)$).
- **Limitações.** O método depende de a binarização e da redução para 41×41 representarem bem a estrutura do labirinto (paredes e corredores de espessuras compatíveis com a grade de 20×20), e de a imagem ter exatamente duas aberturas na borda. A convenção de

que a abertura mais à esquerda é a entrada pode ser trocada sem alterar o restante do programa.

Arquivos entregues

- `main.m`, `id_no.m`, `coordenada_no.m`, `dijkstra.m`, `reconstruir_caminho.m`;
- imagem original do labirinto e imagem binária (`labirinto_binario.png`);
- figuras: `grafo_sobre_labirinto.png`, `caminho_minimo.png` e `grafo_e_caminho.png`.